UNIVERSIDAD DE SAN CARLOS DE GUATEMALA

CENTRO UNIVERSITARIO DE OCCIDENTE

DIVISIÓN DE CIENCIAS DE LA INGENIERÍA

MANEJO E IMPLEMENTACION DE ARCHIVOS

ING BRYAN MONZON



MANUAL TECNICO

MADELAYNE ANA MARIA PEREZ PEREZ

202130171

1. Descripción general

El sistema permite a una organización almacenar, organizar, consultar y compartir archivos de texto (`.txt` y `.md`) entre sus usuarios. Está formado por dos aplicaciones independientes que se comunican mediante una API REST:

- **Cliente:** Angular 21 con Angular Material.
- **Servidor:** Node.js con NestJS (sobre Express) y TypeScript.
- **Base de datos:** MongoDB, accedida solo desde el servidor mediante Mongoose. El contenido de los archivos se guarda en GridFS.

El cliente nunca accede a la base de datos: toda operación pasa por el servidor.

2. Arquitectura

Capa	Tecnología	Puerto	Responsabilidad
Cliente	Angular 21, Angular Material, signals	4200	Interfaz, sesión, consumo de la API
Servidor	NestJS, JWT, bcrypt, class-validator	3000	Reglas de negocio, seguridad, acceso a datos
Base de datos	MongoDB, GridFS	27017	Persistencia de metadatos y contenido

La comunicación es HTTP con JSON y multipart para la subida de archivos. Todas las rutas del servidor usan el prefijo `/api`.

3. Requisitos previos

- Node.js 22.12 o superior (se recomienda 22 LTS o 24 LTS).
- npm 10 o superior.
- MongoDB Community Server en ejecución en `localhost:27017` (servicio `MongoDB` en Windows).
- Angular CLI 21: `npm install -g @angular/cli@21` (o usar `npx ng`).
- Git.

4. Instalación y ejecución

4.1 Obtener el código

```
git clone <url-del-repositorio>
cd gestion-documental
```

4.2 Servidor

```
cd server
npm install
copy .env.example .env
npm run seed
npm run start
```

El servidor queda escuchando en `http://localhost:3000/api`.

4.3 Cliente

En otra terminal:

```
cd client
npm install
ng serve
```

La aplicación queda disponible en `http://localhost:4200`.

4.4 Variables de entorno (`server/.env`)

Variable	Descripción	Ejemplo
`MONGODB_URI`	Cadena de conexión	`mongodb://localhost:27017/gestion_documental`
`JWT_SECRET`	Secreto para firmar los tokens	una cadena larga y aleatoria
`PORT`	Puerto del servidor (opcional)	`3000`

El archivo `.env` no se sube al repositorio; se incluye `.env.example` como plantilla.

4.5 Carga inicial (seed)

`npm run seed` **elimina la base `gestion_documental`** y la recrea con datos reproducibles: 3 usuarios, carpetas y subcarpetas, 50 archivos `.txt` y `.md` almacenados en GridFS, más de 20 comparticiones, recursos en la papelera e historial de actividad con fechas variadas.

Rol	Correo	Contraseña
ADMIN	admin@serviciosprofesionales.com	Admin123!
USER	ana.torres@serviciosprofesionales.com	Ana123!
USER	luis.gomez@serviciosprofesionales.com	Luis123!

5. Estructura del repositorio

```
gestion-documental/
├── server/                                  Aplicación NestJS
│   ├── src/
│   │   ├── auth/                  Login, JWT, guards y estrategia
│   │   ├── users/                 Usuarios, roles y estado
│   │   ├── folders/               Jerarquía de carpetas
│   │   ├── files/                 Archivos y GridFS
│   │   ├── shares/          Comparticiones y permisos (AccessService)
│   │   ├── trash/               Papelera, restauración y purga
│   │   ├── activity/            Historial de actividad
│   │   └── reports/             Reportes con aggregation
│   └── seed/                    Script de carga inicial
├── client/                                  Aplicación Angular
│   └── src/app/
│       ├── core/            api, guards, interceptors, models, services
│       ├── shared/          Tabla reutilizable, diálogos y filtros
│       └── pages/           Una carpeta por pantalla
└── docs/            Modelo documental, API, manuales y diagramas
```

6. Modelo documental (resumen)

El detalle completo, con ejemplos e índices, está en el documento *Modelo documental*.

Colección	Contenido	Decisión principal
`users`	Credenciales, rol, estado, `storageUsed`	Datos 1:1 embebidos; sin arreglos de archivos
`folders`	Jerarquía con `parentId`, `ancestors` y `depth`	Cadena de ancestros embebida (materialized path)
`files`	Metadatos y referencia `gridFsId`	Contenido referenciado en GridFS
`shares`	Permisos recurso-usuario	Colección aparte, relación muchos a muchos
`activity_log`	Auditoría append-only	Copia de `actorName` y `resourceName`
`fs.files` / `fs.chunks`	Contenido binario	Gestionado por GridFS

Índices destacados

- `users`: `{ email: 1 }` único.
- `folders`: único parcial `{ ownerId, parentId, name }` con `isDeleted: false`; multikey `{ ancestors, isDeleted }`.
- `files`: único parcial `{ ownerId, folderId, name }` con `isDeleted: false`; único `{ gridFsId }`; compuestos para reportes por tipo y fecha.
- `shares`: único `{ resourceType, resourceId, granteeId }`.

- `activity_log`: `{ actorId, createdAt }`, `{ resourceType, resourceId, createdAt }`, `{ action, createdAt }`.

7. Reglas de negocio implementadas

7.1 Autenticación y roles

- Contraseñas con bcrypt; sesión mediante JWT enviado como `Authorization: Bearer <token>`.
- Roles `ADMIN` (administra usuarios, ve información global) y `USER` (sus recursos y los compartidos con él).
- Un usuario inactivo no puede iniciar sesión; su historial se conserva.
- El propietario de un recurso siempre se obtiene del token, nunca del cuerpo de la petición.

7.2 Carpetas

- Al crear: `ancestors = [...padre.ancestors, padre._id]` y `depth = padre.depth + 1`.
- Al mover se rechaza el destino si es la misma carpeta o si su cadena `ancestors` contiene a la carpeta movida (evita ciclos). Después se actualizan `ancestors` y `depth` de todos los descendientes con `updateMany` y una pipeline, sin recursión en JavaScript.
- Los nombres son únicos entre carpetas hermanas activas (error 409 si chocan).

7.3 Archivos

- Solo `.txt` y `.md`; se valida la extensión, el tipo y que el contenido sea UTF-8 válido sin bytes nulos. Tamaño máximo de 5 MB.
- Subida en orden seguro (sin transacciones): escribir en GridFS, crear el documento en `files`, incrementar `storageUsed`. Si un paso falla, se compensa borrando lo ya escrito.
- Reemplazar contenido: nuevo binario, actualizar metadatos, ajustar `storageUsed` con la diferencia y borrar el binario anterior.
- Descarga por streaming con el tipo de contenido correcto. El contenido recuperado es idéntico al original.

7.4 Permisos y comparticiones

- Niveles: `READ < COMMENT < EDIT`; el propietario es `OWNER`.
- `AccessService.getEffectiveAccess` calcula el permiso efectivo: para un archivo, el máximo entre su permiso directo y los de su carpeta y de todos los ancestros; para una carpeta, los suyos y los de sus ancestros.
- Mover, eliminar, crear y compartir son exclusivos del propietario. `READ` permite ver y descargar; `EDIT` permite renombrar y reemplazar contenido.
- Sin permiso suficiente el servidor responde 404, para no revelar que el recurso existe.

7.5 Papelera

- Eliminar es un borrado lógico (`isDeleted`, `deletedAt`, `deletedBy`, `deletedRootId`). No toca GridFS ni `storageUsed`.

- Eliminar una carpeta marca su contenido con el mismo `deletedRootId`. Lo que ya estaba en la papelera conserva su propia raíz.
- Restaurar exige que el padre esté activo y que el nombre no choque (409 en caso contrario).
- Eliminación definitiva: primero se escribe el registro de auditoría, luego se borran binarios de GridFS, comparticiones y documentos, y por último se resta `storageUsed` con `$inc`.

7.6 Historial de actividad

- Colección `activity_log` de solo inserción: no existen endpoints para actualizar ni borrar.
- Cada evento guarda quién, qué acción, sobre qué recurso y cuándo, con copia del nombre del actor y del recurso para que la auditoría sea legible aunque el recurso se purgue.

8. API REST

El contrato completo (métodos, parámetros, respuestas y errores) está en el documento *API*. Resumen por módulo:

Módulo	Rutas principales
Auth	`POST /api/auth/register`, `POST /api/auth/login`
Usuarios (ADMIN)	`GET /api/users`, `PATCH /api/users/:id`
Carpetas	`POST/GET /api/folders`, `PATCH /api/folders/:id/rename`, `PATCH /api/folders/:id/move`, `DELETE /api/folders/:id`
Archivos	`POST/GET /api/files`, `GET /api/files/:id`, `GET /api/files/:id/download`, `PATCH .../rename`, `PATCH .../move`, `PUT .../content`, `DELETE /api/files/:id`
Comparticiones	`POST /api/shares`, `PATCH/DELETE /api/shares/:id`, `GET /api/shares/shared-by-me`, `GET /api/shares/shared-with-me`
Papelera	`GET /api/trash`, `POST /api/trash/:type/:id/restore`, `DELETE /api/trash/:type/:id`, `DELETE /api/trash`
Actividad	`GET /api/activity`
Reportes	`GET /api/reports/<reporte>`

Códigos de error comunes

Código	Significado
400	Datos inválidos, ciclo de carpetas, extensión no permitida
401	Falta el token o no es válido
403	Un USER intenta consultar datos de otro usuario en reportes
404	El recurso no existe o no hay permiso

Código	Significado
409	Conflicto de nombre o padre en la papelera
413	Archivo mayor al límite

9. Reportes

Todos se calculan con aggregation pipelines de MongoDB, sin procesar datos en JavaScript. Un ADMIN ve datos globales; un USER solo los suyos.

1. Archivos por tipo.
2. Archivos en un rango de fechas.
3. Archivos con más operaciones.
4. Top 10 de usuarios con más archivos.
5. Usuarios con más recursos compartidos.
6. Archivos compartidos con más usuarios.
7. Recursos compartidos por un usuario.
8. Recursos recibidos por un usuario.
9. Archivos que nunca han sido compartidos.
10. Carpetas con más archivos.
11. Historial de un usuario en un rango de fechas.
12. Operaciones por tipo.
13. Operaciones por período.
14. Principales acciones.

10. Cliente Angular

- Componentes standalone, rutas con carga perezosa (`loadComponent`) y guards `authGuard` y `adminGuard`.
- Un interceptor agrega el token a cada petición y cierra la sesión ante un 401.
- Una sola tabla reutilizable (`DataTableComponent`) se usa en todas las pantallas; los diálogos de texto y de confirmación también son reutilizables.
- Los servicios de `core/services` encapsulan las llamadas HTTP por dominio.

11. Seguridad

- Contraseñas con hash bcrypt; `passwordHash` nunca se devuelve por la API.
- Validación de entradas con `class-validator` y `ValidationPipe` global con lista blanca.
- Control de acceso por rol y por permiso sobre cada recurso.
- CORS limitado a `http://localhost:4200`.
- Secretos en `.env`, fuera del repositorio.

12. Consistencia sin transacciones

MongoDB se ejecuta como instancia independiente, sin transacciones. Para mantener la consistencia, las operaciones aditivas escriben el binario primero y compensan si fallan; las destructivas registran la auditoría antes de borrar y son idempotentes, por lo que pueden reintentarse.

13. Pruebas realizadas

Cada módulo se verificó con `npm run build`, `npm run lint` y peticiones HTTP reales, contrastando resultados con consultas directas en `mongosh`. Se comprobó, entre otras cosas, que la descarga de un archivo es idéntica a la subida, que `storageUsed` coincide con la suma de `files.size`, que no quedan binarios huérfanos en GridFS tras purgar y que un ciclo de carpetas es rechazado.

14. Solución de problemas

Problema	Solución
El servidor no conecta a MongoDB	Verificar que el servicio esté activo (`sc query MongoDB`) y revisar `MONGODB_URI`
`ng` no se reconoce	Instalar el CLI (`npm install -g @angular/cli@21`) o usar `npx ng serve`
Error de versión de Node	Usar Node 22.12 o superior
El login falla tras cambiar datos	Volver a ejecutar `npm run seed`
Error de CORS	Abrir el cliente en `http://localhost:4200`

15. Uso de herramientas de apoyo

Durante el desarrollo se utilizó OpenCode como asistente, con la autorización del docente y del auxiliar del curso.